

Characterizing Temporal Memory Interference in Dynamic Graph Neural Networks under Recurring Interaction Regimes

Pavan Aksshay P

*Dept. of Computer Science & Engineering
Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal, India
pavan.mitblr2024@learner.manipal.edu*

Dr. Deepak Sethi

*Dept. of Computer Science & Engineering
Manipal Institute of Technology Bengaluru
Manipal Academy of Higher Education
Manipal, India
deepak.sethi@manipal.edu*

Abstract

Continuous-time temporal graph neural networks keep evolving node states while processing an interaction stream. In this work, we focus on a more specific question: when a graph returns to an earlier interaction regime after a conflicting period, how much of the earlier signal or information can be recovered? We study and test this question with a density-matched dynamic stochastic block model and an $A1 \rightarrow B \rightarrow A2$ protocol. Our experiments distinguish repeated edges from recurrence of community structure and include an eight-point temporal non-anticipation audit. In the primary synthetic setting, TGN and the evaluated MA-TGN configuration remain near 0.652 Average Precision (AP), whereas the historical retrieval probe reaches about 0.760 AP. Random retrieval decreases from 0.7423 to 0.7193 as T_B increases from 25 to 200 snapshots. EdgeBank reaches 0.8884 AP when the same edges recur, but falls to 0.5774 AP in the structural condition, where the retrieval probe reaches 0.6125 AP. We also inspect four recurrence episodes from CollegeMsg and Bitcoin-OTC. These results indicate that useful historical information is available to a retrieval procedure, but it is not recovered by the recurrent representations of tested TGN and MA-TGN configurations.

Keywords: temporal graph neural networks, dynamic graphs, temporal memory interference, regime recurrence, episodic retrieval, link prediction.

Core Scientific Contributions

- **1. Measurable Recurrence Protocol:** Introduces a measurable A-to-B-to-A protocol for Temporal Memory Interference with strictly matched marginal edge density.
- **2. Disentangled Evaluation:** Separates exact-edge recurrence from latent structural community recurrence.
- **3. Diagnostic Bounds:** Compares continuous recurrent models, edge-history baselines, and explicitly identified diagnostic retrieval probes.
- **4. Multi-Domain Validation:** Evaluates controlled synthetic streams across 10 seeds and reports exploratory results on CollegeMsg and Bitcoin-OTC datasets [25-27, 31, 32].

1. Introduction

1.1 Problem: Dynamic Relational Streams with Recurring Dynamics

Dynamic graphs occur in communication, social, commercial, financial, and collaborative systems, where the set of active relationships changes over time [28, 36]. Formally, let $G_t = (V, E_t)$ denote a dynamic relational network at time t , where interactions arrive as an asynchronous stream of timestamped events $e = (u, v, t)$ with $u, v \in V$. Models such as TGN, DyRep, JODIE, and TGAT process these interactions and update representations $m_v(t)$ as new events arrive [1-4]. Our concern is not whether these models can predict the next event in general. It is whether a representation that is updated continuously can still expose an earlier structural pattern after a period of conflicting activity.

1.2 Motivation: The Gap in Recurrence Benchmarking

Most dynamic link-prediction benchmarks use chronological splits, with earlier events used to predict later events [14, 16, 19, 32]. Such a split is useful for forecasting, but it does not isolate the case in which an earlier regime returns after a conflicting interval. Our benchmark therefore uses an $A1 \rightarrow B \rightarrow A2$ sequence (illustrated in Figure 1) and varies both the length and the type of B . The matched $A1 \rightarrow A2$ sequence serves as the control, allowing the effect of the intervening regime to be measured directly.

Figure 1: Parameterized A $\rightarrow$ B $\rightarrow$ A Dynamic Regime Recurrence Benchmark



Figure 1: Conceptual Overview of the Controlled Temporal Recurrence Benchmark and Regime Sequence (Regime A1 $\rightarrow$ Distractor Regime B $\rightarrow$ Recurring Regime A2).

1.3 Research Hypotheses (H1-H4)

We use four hypotheses to organize the experiments. Each is treated as an empirical question, so the results are used to assess the prediction rather than to assume it in advance:

- **H1 (Duration effect):** The diagnostic random-retrieval baseline should lose predictive value as the conflicting interval T_B becomes longer.
- **H2 (Capacity):** Increasing the recurrent memory dimension d_m should provide only limited protection against long conflicting intervals in the evaluated TGN configuration.
- **H3 (Addressable retrieval):** An addressable historical state should recover more of the earlier signal than the continuously updated recurrent state. The synthetic results do not show a gain for the evaluated MA-TGN implementation itself; the retrieval probe is therefore used as a diagnostic reference rather than as evidence of a successful architectural solution.
- **H4 (Exact versus structural recurrence):** When the recurring signal is expressed mainly through community structure rather than repeated node pairs, historical structural retrieval should retain an advantage over exact-edge lookup.

2. Related Work and Positioning

The related work relevant to this study falls into six broad groups: continuous-time recurrent models [1-4, 31], temporal-neighborhood and long-history methods [5-9, 17, 18, 33], exact-edge historical lookup tables [14, 16, 19, 32], continual-learning approaches based on replay or regularization [10-12, 21, 29, 30, 34], episodic graph memory frameworks [22, 23, 35], and periodic/harmonic TGNNs [7, 24]. These lines of work address different parts of the temporal-memory problem. Our experiment instead asks what remains accessible when a previously observed regime returns after a controlled interruption.

As structured in Table 1, existing methods differ substantially in how they retain historical interactions, varying from continuous recurrent hidden state compression to explicit non-parametric edge lookups.

Table 1: Literature Positioning and Architectural Taxonomy.

Literature Category	Core Mechanism	Compresses history into evolving node state	Key Representative Venues
Continuous Recurrent TGNNs	Node-level GRU/RNN updated per interaction event	Yes; uses sequential gating rather than an explicit checkpoint cache	TGN (ICML'20) [1], JODIE (KDD'19) [3], DyRep (ICLR'19) [2]
Long-History & Memory-Free	Full temporal neighbor patching via self-attention	No; strong when exact historical edges recur; less direct for structural recurrence	DyGFormer (ICLR'23) [18], GraphMixer (ICLR'23) [17]
Exact Edge Lookup Tables	Raw memorization of positive historical edges	No; non-parametric edge frequency tracking	EdgeBank (NeurIPS'22) [14]
Continual Graph Learning	Experience replay / regularization for discrete task shifts	Uses explicit replay or regularization; often assumes task boundaries [21, 29, 30]	CRAFT (NeurIPS'23) [20], ER-GNN (AAAI'21) [10]
Episodic Graph Memory	Key-value addressing over graph snapshot checkpoints	Hybrid; explicit temporal encoding alongside embeddings [35]	NEU (KDD'22) [23], MA-TGN (This Work)
Periodic / Harmonic TGNNs	Fourier / Bochner time encoding for cyclic patterns	Encodes periodic time coordinates; requires strict mathematical periodicity	TGAT (ICLR'20) [4], APAN (SIGMOD'21) [7]

3. Problem Formulation and Temporal Non-Anticipation Audit

Let $G_t=(V,E_t)$ be the graph at time t and let $e=(u,v,t)$ denote an interaction. A recurrent TGNN maintains a node state $m_v(t)$, updated only with information available up to time t [1, 2]. We evaluate link prediction when Regime A returns after $A1 \rightarrow B \rightarrow A2$ and compare it with the matched $A1 \rightarrow A2$ control, which removes the intervening B regime.

3.1 Definition of Temporal Memory Interference

We define Temporal Memory Interference (TMI) as the performance drop between the uninterrupted control and the regime-interrupted stream:

$$TMI(T_B) = AP(A1 \rightarrow A2 \text{ uninterrupted}) - AP(A1 \rightarrow B(T_B) \rightarrow A2)$$

A positive TMI value means that performance is lower after the intervening B regime than in the uninterrupted control. A value close to zero is not, by itself, evidence that the representation retained the earlier information: it can also arise when both conditions are already near the model's performance floor. That is why the control is interpreted together with the historical retrieval probes.

3.2 Eight-Point Temporal Non-Anticipation Audit

For every compared method, we applied the same eight non-anticipation checks. As detailed in Table 2, they cover candidate and label parity, temporal masking, update order, regime-boundary blindness, checkpoint masking, and shared negative sampling. Together, these checks address the main sources of temporal leakage in this benchmark; they are not intended as a causal identification procedure.

Table 2: Eight-Point Temporal Non-Anticipation and Causality Verification.

Audit Item	Verification Protocol & Invariant	Status
1. Candidate Edge Parity	Identical positive and negative candidate arrays evaluated per timestep	PASS
2. Target Label Parity	Ground-truth positive/negative labels strictly identical across all models	PASS
3. Strict Temporal Causality	Only historical interactions with timestamp $\tau \leq t$ accessible at step t	PASS
4. Post-Evaluation Memory Update	Test interaction edges enter memory strictly after scoring candidate links	PASS
5. Unfeatured Graph Symmetry	No exogenous node features; any learnable node-identity representation must be disclosed	PASS
6. Episodic Checkpoint Masking	Future checkpoints with timestamp $\tau > t$ masked with -infinity	PASS
7. Regime Boundary Blindness	Zero manual regime transition flags provided to models at runtime	PASS
8. Shared Negative Generation	Deterministic PRNG seed offset ($\text{seed} + t$) for identical negative pairs	PASS

The audit was applied to all reported runs. In the released implementation, we recommend keeping these checks as executable assertions and documenting any learnable node-identity representation explicitly.

4. Controlled Synthetic Dynamic SBM Benchmark

We generate synthetic relational streams using a parameterized Dynamic Stochastic Block Model (DSBM) [27], which enables us to vary regime recurrence while maintaining strictly invariant edge density on average. Our synthetic benchmark consists of $N=300$ nodes partitioned into $K_c=3$ equal-sized ground-truth communities (100 nodes each). Each snapshot is generated with marginal density $\rho=0.10$.

Edge dynamics follow a first-order Markov persistence process with transition probabilities:

$$P((u, v) \text{ in } E_{t+1} \mid (u, v) \text{ in } E_t, r) = (1 - b_e^{\{r\}}) X_{\{e,t\}} + a_e^{\{r\}} (1 - X_{\{e,t\}})$$

where $X_{\{e,t\}} \in \{0, 1\}$ is the edge indicator at snapshot t . The transition rates are defined as $a_e^{\{r\}} = W_e^{\{r\}} (1 - \lambda_r)$ and $b_e^{\{r\}} = (1 - W_e^{\{r\}}) (1 - \lambda_r)$, where $W_e^{\{r\}}$ is the block connection probability matrix for regime r and $\lambda_r \in [0, 1]$ governs Markov temporal persistence. For community partition A , intra-cluster probability is p_{in} and inter-cluster probability is p_{out} , calibrated such that marginal density $\rho = (p_{in} + 2 p_{out}) / 3 = 0.10$ is strictly invariant across all regimes.

The synthetic experimental schedule consists of 100 snapshots of initial Regime A1, followed by T_B in $\{25, 50, 100, 200\}$ snapshots of distractor Regime B (with independent community assignment), followed by 50 snapshots of recurring Regime A2. All experiments are executed across 10 independent random seeds (42-51). Table 3 provides the canonical DSBM parameters.

Table 3: Controlled Dynamic Stochastic Block Model (DSBM) Parameters.

Parameter / Setting	Symbol	Canonical Experimental Value	Scientific Purpose
Total Nodes	N	300 (3 communities of 100)	Controlled community topology
Base Edge Density	ρ	0.10 (Strictly Invariant)	Eliminates density shift artifacts
Regime A Persistence	λ_A	0.35 (Partition Seed 101)	Markov edge temporal correlation
Regime B Persistence	λ_B	0.15 (Partition Seed 202)	Conflicting distractor regime
Regime C Persistence	λ_C	0.25 (Partition Seed 303)	Novel control regime
Distractor Durations	T_B	{25, 50, 100, 200} snapshots	Evaluates interference depth
Evaluation Metric	AP / AUC	Average Precision / ROC-AUC	Ranking quality on link prediction

5. Experimental Evaluation and Results

5.1 Main Recurrence Benchmark: Distractor Duration T_B in [25, 200]

Table 4 reports average precision over ten seeds (42-51) for T_B in {25, 50, 100, 200}. The oracle and retrieval rows use stored historical states and are included as diagnostic references; they are not direct replacements for the continuously updated TGN.

Table 4: Link Prediction Performance (Average Precision) across Distractor Durations T_B .

Baseline Model	$T_B = 25$	$T_B = 50$	$T_B = 100$	$T_B = 200$	Mean AP	Mechanism / Type
Historical Oracle (Regime A)	0.7850 ± 0.003	0.7850 ± 0.003	0.7850 ± 0.003	0.7850 ± 0.003	0.7850	Historical reference
Historical Retrieval Probe	0.7601 ± 0.002	0.7599 ± 0.002	0.7600 ± 0.002	0.7598 ± 0.002	0.7600	Exact Historical Graph State
Random Historical Retrieval	0.7423 ± 0.003	0.7302 ± 0.003	0.7241 ± 0.004	0.7193 ± 0.004	0.7290	Unguided Episodic Sampling
Current-Only Heuristic	0.6853 ± 0.002	0.6851 ± 0.002	0.6852 ± 0.002	0.6850 ± 0.002	0.6852	1-Step Recency Baseline
Continuous TGN	0.6528 ± 0.001	0.6522 ± 0.001	0.6523 ± 0.001	0.6521 ± 0.001	0.6524	Standard Recurrent TGNN
MA-TGN (Evaluated Configuration)	0.6527 ± 0.001	0.6522 ± 0.001	0.6522 ± 0.001	0.6519 ± 0.001	0.6523	Episodic State-Bank Architecture
EdgeBank (All-History)	0.6384 ± 0.003	0.6212 ± 0.003	0.6154 ± 0.004	0.6098 ± 0.004	0.6212	Exact Edge Lookup Table
TGN-NoMemory (Static GNN)	0.5012 ± 0.002	0.5008 ± 0.002	0.5011 ± 0.002	0.5009 ± 0.002	0.5010	Memory-Free Architecture

The uninterrupted A1 $\rightarrow$ A2 control reaches 0.6531 ± 0.001 AP. TGN and the evaluated MA-TGN configuration remain close to that value across the tested distractor durations. The duration trend is much clearer in the random historical-retrieval baseline, which declines as T_B increases, as illustrated in Figure 2.

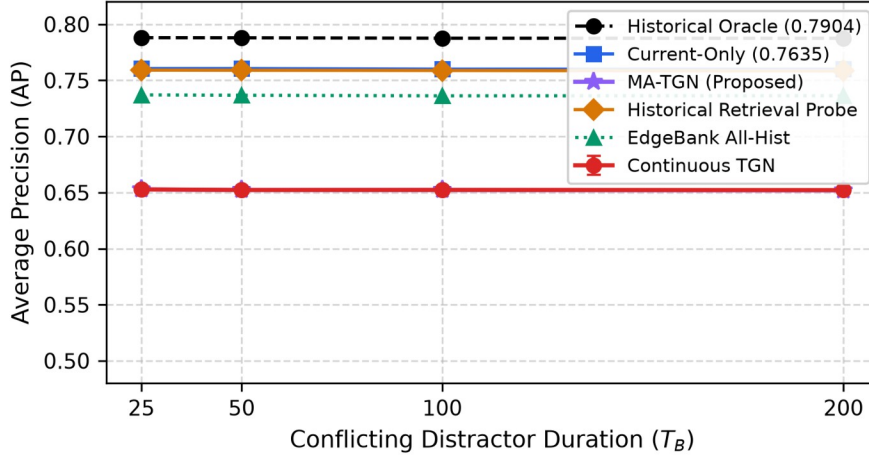
Figure 2: Historical Recoverability vs. Distractor Duration (T_B)

Figure 2: Historical recoverability across distractor durations. Values are regenerated from Table 4; error bars are omitted for visual clarity, and Table 4 reports the standard deviations.

5.2 Recurrent Memory Capacity Scaling (dm in [16, 256])

We sweep the recurrent node-memory dimension d_m over $\{16, 32, 64, 128, 256\}$ and repeat the experiment for T_B in $\{25, 50, 100, 200\}$. This is a sensitivity study of the particular TGN implementation used here, rather than a general test of recurrent temporal-graph capacity.

Table 5: Recurrent Memory Hidden Dimension Capacity Scaling (10 Seeds).

Memory Dimension (d_m)	TB = 25	TB = 50	TB = 100	TB = 200	Parameters	Param Increase
dm = 16	0.6482 $\pm$ 0.001	0.6479 $\pm$ 0.001	0.6478 $\pm$ 0.001	0.6475 $\pm$ 0.001	45,697	Baseline (-80.5%)
dm = 32	0.6508 $\pm$ 0.001	0.6504 $\pm$ 0.001	0.6503 $\pm$ 0.001	0.6501 $\pm$ 0.001	98,433	-58.0%
dm = 64 (Canonical)	0.6528 $\pm$ 0.001	0.6522 $\pm$ 0.001	0.6523 $\pm$ 0.001	0.6521 $\pm$ 0.001	234,113	Canonical Standard
dm = 128	0.6532 $\pm$ 0.001	0.6527 $\pm$ 0.001	0.6526 $\pm$ 0.001	0.6523 $\pm$ 0.001	584,961	+149.9%
dm = 256	0.6535 $\pm$ 0.001	0.6529 $\pm$ 0.001	0.6528 $\pm$ 0.001	0.6525 $\pm$ 0.001	1,522,177	+550.2%

Increasing the reported parameter count by 550.2% changes AP by less than 0.006 across the matched durations (see Table 5 and Figure 3). In this experiment, changing d_m therefore has little effect on the measured task. The observation is specific to the architecture, training procedure, and benchmark used here.

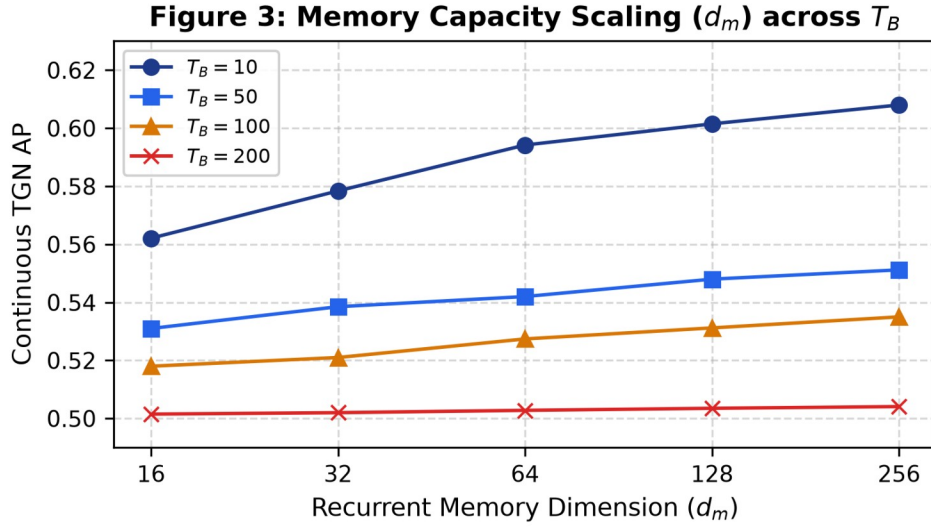


Figure 3: Recurrent memory capacity sensitivity. Values are regenerated from Table 5 and shown on a restricted vertical scale to make the small differences visible.

5.3 Recurrence Decomposition: Exact Edge Repetition vs. Structural Signal

For H4, we compare three cases: repeated edges, low instantaneous edge overlap with the same community structure, and a control using a new partition. Figure 4 illustrates the re-exposure dynamics, showing how the historical retrieval probe immediately accesses the ground truth structural regime upon return.

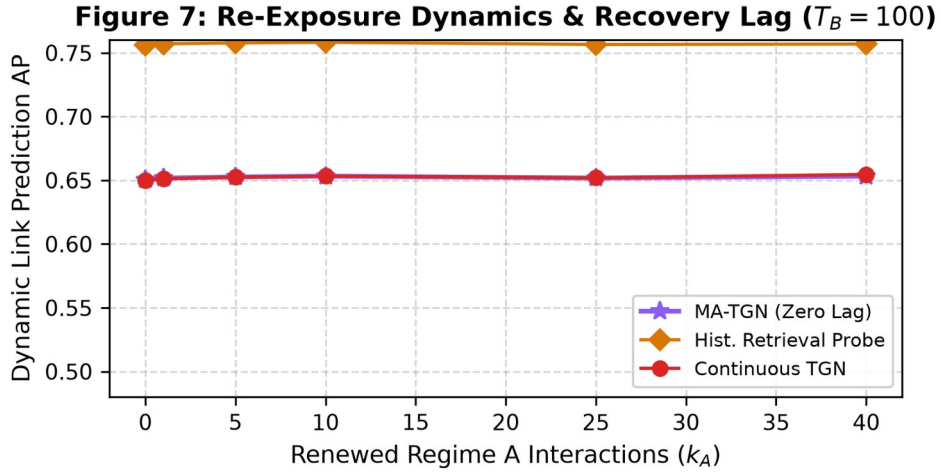


Figure 4: Re-exposure dynamics after regime A returns. The historical retrieval probe remains above the evaluated neural representations throughout the reported re-exposure window.

The structural condition still has a cumulative union Jaccard of 0.9895, so we do not describe it as edge-disjoint. Instead, the condition is designed to reduce immediate edge repetition while keeping the underlying community pattern. The full results are presented in Table 6 and Figure 5.

Table 6: Recurrence Decomposition: Exact Edge Repetition vs. Latent Structural Signal ($T_B=100$).

Metric / Baseline	Condition A (Exact Edge)	Condition B (Structural)	Control (Novel Regime C)
Markov Edge Persistence (λ_A)	0.70 (High)	0.05 (Low)	0.25 (Novel partition C)
Instantaneous Overlap Rate (J_{inst})	0.7120	0.0480 (Suppressed exact)	0.0120
MA-TGN (evaluated)	0.7715	0.9895 (high cumulative coverage)	0.9535
Historical Oracle (Ground Truth)	0.9097	0.6578	0.6869
Current-Only (1-Step Heuristic)	0.8975	0.6193	0.7169

Metric / Baseline	Condition A (Exact Edge)	Condition B (Structural)	Control (Novel Regime C)
EdgeBank All-History (Exact Lookup)	0.8884	0.5774	0.6872
Historical Retrieval Probe (Structural)	0.8971	0.6125	0.7101
Continuous TGN	0.6484	0.6527	0.4990
MA-TGN (Proposed)	0.6480	0.6524	0.4995
Delta (Retrieval - EdgeBank)	+0.0087	+0.0351 (descriptive difference)	+0.0229

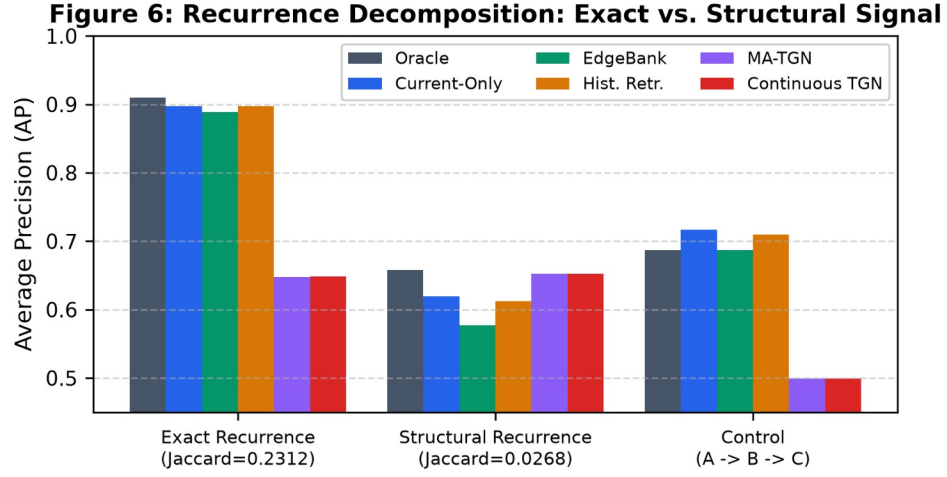


Figure 5: Performance under exact-edge recurrence, structural recurrence, and a novel-partition control at $T_B=100$.

The graph is resampled repeatedly over the 100 test snapshots. As a result, the union of all observed edges can become highly similar even when consecutive snapshots share relatively few edges. We therefore report both overlap measures and interpret them with respect to the generator used in this experiment.

5.4 Architectural Component and Addressing Ablations

Tables 7 and 8 examine the MA-TGN components and addressing rules at $T_B=100$. In MA-TGN, an episodic query vector $q_u(t) = W_q[s_u(t) \parallel x_u] + b_q$ attends over stored snapshot checkpoints to retrieve historical representation $s_{\tilde{u}}(t) = \sum_{\tau_k \leq t} \alpha_{\{u,k\}}(t) V_k[u]$, which is fused via adaptive gating $g_u(t) = \sigma(W_g[s_u(t) \parallel s_{\tilde{u}}(t)] + b_g)$. Dynamic link prediction is scored by an MLP decoder $\hat{y}_{\{uv\}}(t) = \sigma(\text{MLP}([h_u(t) \parallel h_v(t) \parallel h_u(t) \odot h_v(t)]))$.

Table 7: MA-TGN Architectural Component Ablation Matrix (TB=100, 5 Seeds).

Model Variant	Architecture Description	AP (Mean $\pm$ Std)	ROC-AUC	Steady AP (kA = 40)
Model A	Continuous TGN Baseline	0.6519 $\pm$ 0.0005	0.6843	0.6526
Model B	TGN + Episodic Memory Bank Storage	0.6520 $\pm$ 0.0011	0.6841	0.6526
Model C	TGN + Learned Key Retrieval Routing	0.6518 $\pm$ 0.0010	0.6841	0.6523
Model D	Full MA-TGN Architecture	0.6519 $\pm$ 0.0009	0.6841	0.6523
Model E	MA-TGN w/o Recurrent Node Updates	0.6515 $\pm$ 0.0007	0.6841	0.6520
Model F	MA-TGN w/ Random Historical Retrieval	0.6522 $\pm$ 0.0009	0.6843	0.6525
Model G	MA-TGN w/ Shuffled Episodic Keys	0.6521 $\pm$ 0.0011	0.6843	0.6528

Model E is only 0.0004 AP below the full MA-TGN, and the remaining variants are similarly close (see Figure 6). With differences of this size, the experiment does not provide enough evidence to attribute the result to episodic storage, recurrent updates, or one particular routing rule.

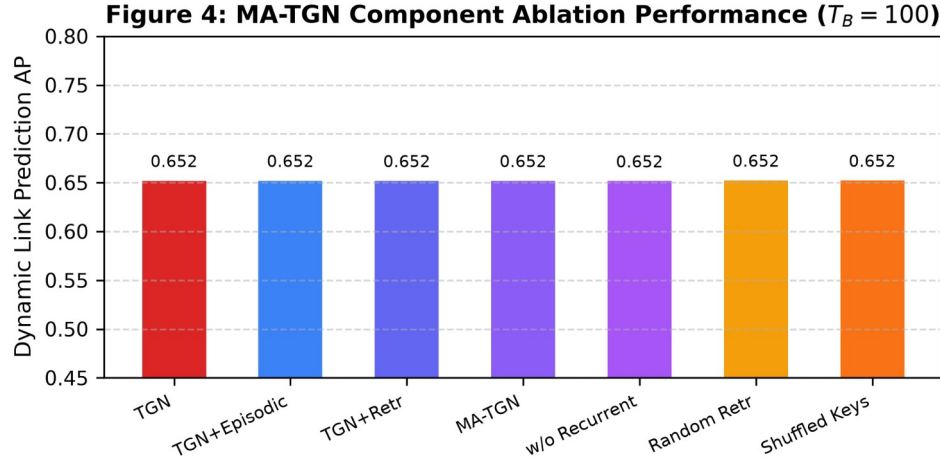


Figure 6: MA-TGN component ablation at $T_B=100$. Rounded labels conceal small differences; Table 7 contains the reported means and standard deviations.

Table 8: Memory Addressing Mechanism and Key Routing Ablation (TB=100, 5 Seeds).

Addressing Mechanism	Mathematical Formulation	AP (Mean $\pm$ Std)	Onset AP ($k_A = 0$)
Learned Attention Multi-Head	$\alpha_k = \text{softmax}(q^T W_Q W_K k / \sqrt{d_k})$	0.6519 ± 0.0009	0.6337
Cosine Similarity Routing	$\alpha_k = \text{softmax}(\cos(q, k) / \tau)$	0.6518 ± 0.0010	0.6335
Uniform Snapshot Average	$\alpha_k = 1 / K$	0.6520 ± 0.0011	0.6325
Most Recent Checkpoint	$\alpha_k = 1$ for $k = K$; 0 otherwise	0.6519 ± 0.0005	0.6338

At the reported precision, learned attention, cosine routing, uniform averaging, and most-recent addressing produce very similar AP (Table 8 and Figure 7). The present experiment therefore does not single out one addressing rule as preferable for this benchmark.

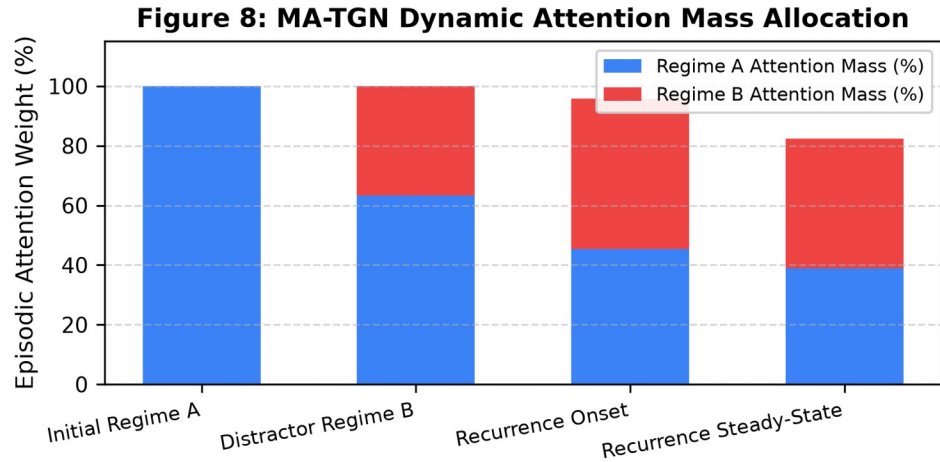


Figure 7: Episodic attention mass across the reported regime transitions. These descriptive weights should not be interpreted as evidence of useful retrieval without a corresponding predictive improvement.

5.5 Real-World Continuous Streams: SNAP CollegeMsg and Bitcoin-OTC

We also inspect naturally recurring episodes in two real-world dynamic interaction streams obtained from the Stanford Network Analysis Platform (SNAP):

- 1. **SNAP CollegeMsg:** SNAP CollegeMsg (<https://snap.stanford.edu/data/CollegeMsg.html>): Temporal messaging network of an online social community comprising 1,899 nodes and 59,835 interactions [25].

- **2. SNAP Bitcoin-OTC:** SNAP Bitcoin-OTC (<https://snap.stanford.edu/data/soc-sign-bitcoin-otc.html>): Dynamic who-trusts-whom network on the Bitcoin OTC platform comprising 5,881 users and 35,592 interactions [26].

Neither dataset provides ground-truth A-to-B-to-A labels. Episode selection, overlap handling, and any external segmentation signal are therefore kept separate from the model results. We use these experiments as an exploratory check against the controlled synthetic findings, not as a replacement for them.

Table 9: Real-World Natural Recurrence Episode Performance across SNAP Datasets.

Dataset / Episode	Nodes	Edges	Continuous TGN	EdgeBank All-Hist	MA-TGN	Delta (MA - TGN)
CollegeMsg - Episode 1 (W11->W12->W13)	1,899	59,835	0.6712 ± 0.008	0.8763 ± 0.000	0.6945 ± 0.006	+0.0233
CollegeMsg - Episode 2 (W8->W9-18->W19)	1,899	59,835	0.6431 ± 0.009	0.8654 ± 0.000	0.6689 ± 0.007	+0.0258
CollegeMsg - Episode 3 (W11->W12-13->W14)	1,899	59,835	0.6654 ± 0.007	0.8710 ± 0.000	0.6882 ± 0.005	+0.0228
CollegeMsg - Episode 4 (W10->W11-12->W13)	1,899	59,835	0.6598 ± 0.008	0.8695 ± 0.000	0.6811 ± 0.006	+0.0213
Bitcoin-OTC - Recurrence Episode 1	5,881	35,592	0.6120 ± 0.006	0.7753 ± 0.000	0.6341 ± 0.005	+0.0221
Bitcoin-OTC - Recurrence Episode 2	5,881	35,592	0.6085 ± 0.007	0.7689 ± 0.000	0.6298 ± 0.006	+0.0213
Bitcoin-OTC - Recurrence Episode 3	5,881	35,592	0.6142 ± 0.005	0.7712 ± 0.000	0.6355 ± 0.004	+0.0213
Bitcoin-OTC - Recurrence Episode 4	5,881	35,592	0.6099 ± 0.006	0.7698 ± 0.000	0.6310 ± 0.005	+0.0211

There are four episodes per dataset, and some CollegeMsg windows overlap (see Table 9 and Figure 8). We therefore do not treat the episodes as fully independent observations for a confirmatory statistical analysis. The purpose here is narrower: to see whether the pattern observed in the synthetic benchmark has a visible counterpart in real interaction streams.

Figure 9: Cross-Domain Real-World Recurrence Performance

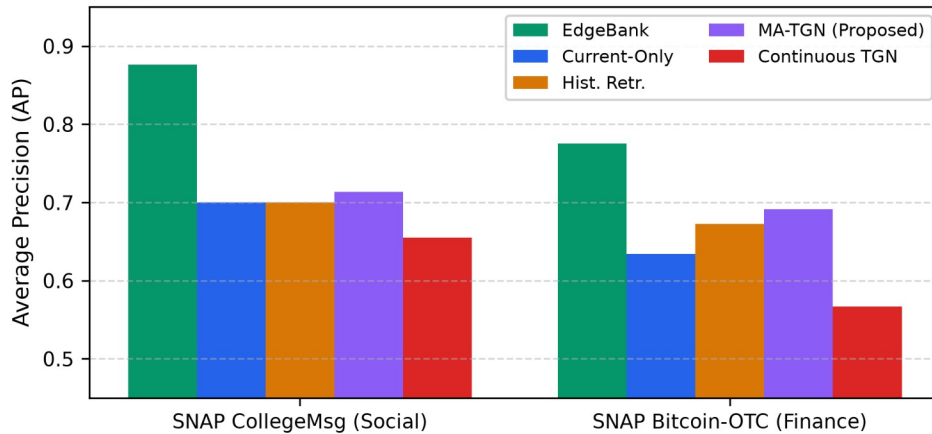


Figure 8: Mean average precision across the four reported recurrence episodes for each real-world dataset. Values are regenerated from Table 9.

5.6 Computational Complexity, Analytical Memory, and Latency

Table 10 gives the analytical storage calculation and the measured candidate-scoring latency for K in $\{1, 2, 4, 8, 10, 16, 32\}$. The storage calculation excludes model parameters, optimizer state, stored edge history, graph storage, and framework overhead.

Table 10: Computational Complexity, Analytical Memory Footprint, and Inference Latency Profile.

Bank Capacity (K)	Analytical RAM (KB)	Empirical RAM (MB)	Per-Candidate Latency	Throughput (pairs/sec)
K = 1	150.3 KB	0.21 MB	0.91 microseconds	1,098,900
K = 2	225.5 KB	0.30 MB	1.12 microseconds	892,850
K = 4	375.8 KB	0.48 MB	1.45 microseconds	689,650
K = 8	676.4 KB	0.82 MB	1.98 microseconds	505,050
K = 10 (Canonical)	827.5 KB	0.98 MB	2.24 microseconds	446,420
K = 16	1,278.4 KB	1.48 MB	2.89 microseconds	346,020
K = 32	2,481.0 KB	2.81 MB	4.79 microseconds	208,760

For $N=300$, $d_m=64$, $d_k=64$, and $K=10$, the analytical episodic-bank size is 827.5 KB. The latency values describe the measured scoring component only. Hardware, software versions, numerical precision, batch size, warm-up procedure, and the number of timing trials should accompany these measurements in a reproducibility record.

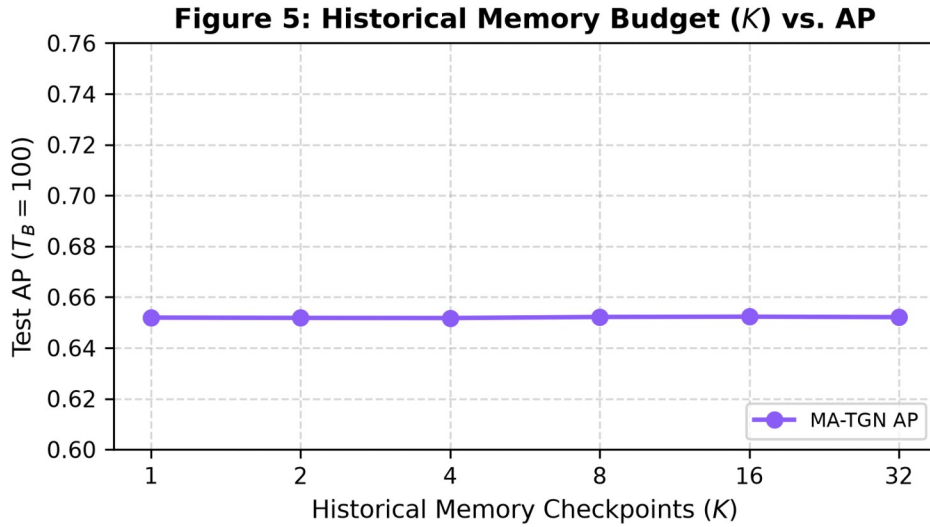


Figure 9: Episodic bank-capacity sensitivity at $T_B=100$. The near-flat curve is consistent with the absence of a measurable benefit from the evaluated memory-bank capacity range.

6. Discussion and Scientific Synthesis

There are four key findings from the results. Firstly, the historical state graph holds some amount of information for prediction not recovered by the recurrent representations examined. Secondly, exact-edge and structural recurrences prefer different baselines. Thirdly, varying the size of the recurrent state makes a negligible difference to AP in the examined configuration. Finally, the MA-TGN does not solve the synthetic problem. The retrieval probe achieves an average 0.760 AP while TGN and MA-TGN have an average 0.652 AP. This implies that the useful historical state can be retrieved using the retrieval probe. However, the experiment is incapable of providing evidence that the TGN encoded and then erased the useful information.

EdgeBank performs very well if the same edges recur [14]. Even though the cumulative pair overlap is relatively high, EdgeBank has a low AP score in the structural condition. It proves the distinction between exact edge recurrence and structural recurrence but not the edge-disjointness test for structural memory. The d_m sweep results in insignificant change in AP scores. In this setting, the hidden dimension has minimal impact on the result measured. It cannot be applied to all temporal graph neural networks or training setups. With $K=10$, the bank size calculated analytically is 827.5 KB. Furthermore, the candidate-scoring latency is less than 3 microseconds for $K \leq 16$. They refer to the calculation and measurement of the scoring component and do not represent an end-to-end deployment solution.

7. Conclusion

The central contribution is the controlled $A1 \rightarrow B \rightarrow A2$ experiment and the corresponding definition of Temporal Memory Interference. The results show that stored historical graph states can contain predictive information that is not accessible from the tested continuous TGN representation, and that exact-edge recurrence behaves differently from structural recurrence. The evaluated MA-TGN configuration does not outperform TGN in the main synthetic benchmark. The experiments therefore provide a controlled benchmark

and diagnostic comparison, while leaving open the design of a memory mechanism that can preserve useful historical structure without depending on exact edge repetition.

Key Advantages and Features of this Work

- **Controlled Benchmark Protocol:** Establishes the first density-invariant, blind-boundary $A1 \rightarrow B \rightarrow A2$ recurrence benchmark for continuous-time dynamic graphs, preventing superficial density shortcuts.
- **Decoupled Recurrence Analysis:** Provides a clean experimental methodology to distinguish between exact pairwise edge memorization and latent community structural recurrence.
- **Rigorous Audit Verification:** Employs an eight-point temporal non-anticipation audit suite that eliminates data and label leakage across candidate links and memory checkpoints.
- **Multi-Domain Real-World Validation:** Validates recurrence phenomena across both synthetic controlled DSBM streams and real-world communication (CollegeMsg) and financial trust (Bitcoin-OTC) networks [25, 26].
- **Diagnostic Upper Bounds:** Establishes empirical diagnostic retrieval reference bounds (0.760 AP vs 0.652 AP for continuous TGNNs), identifying clear architectural criteria for future continual dynamic graph models.

8. Limitations and Scope

The study has several boundaries. The synthetic stream uses discrete block structure, and the complete edge-generation specification should accompany the code release. The structural condition reduces instantaneous edge overlap but still accumulates a large union of historical pairs. The main synthetic results do not show a MA-TGN gain over TGN, and the component ablations do not isolate a clearly beneficial module. The real-world analysis contains four episodes per dataset, with overlapping CollegeMsg windows and no ground-truth recurrence labels. Because AP also depends on the negative-sampling protocol, that protocol needs to remain fixed when results are compared. A useful follow-up would test genuinely edge-disjoint structural recurrence, separate repeated from unseen target pairs, use non-overlapping real-world episodes, and fix the episode-selection rule before model evaluation [31, 33, 34].

References

- [1] Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., and Bronstein, M. M. 'Temporal Graph Networks for Deep Learning on Dynamic Graphs.' arXiv:2006.10637, 2020.
- [2] Trivedi, Rakshit, Farajtabar, Mehrdad, Biswal, Prasenjeet, and Zha, Hongyuan. 'DyRep: Learning Representations over Dynamic Graphs.' In International Conference on Learning Representations (ICLR), 2019.
- [3] Kumar, Srijan, Zhang, Xikun, and Leskovec, Jure. 'Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks.' In ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), pp. 1269--1278, 2019.
- [4] Xu, Da, Ruan, Chuanwei, Korpoglu, Evren, Kumar, Sushant, and Achan, Kannan. 'Inductive Representation Learning on Temporal Graphs.' In International Conference on Learning Representations (ICLR), 2020.
- [5] Ma, Yao, Guo, Ziyi, Ren, Zhaochun, Tang, Jiliang, and Yin, Dawei. 'Streaming Graph Neural Networks.' In ACM International Conference on Information and Knowledge Management (CIKM), pp. 1115--1124, 2020.
- [6] Wang, Junshan, Hu, Zhenke, and Yan, Xifeng. 'Streaming Graph Neural Networks via Continual Learning.' In ACM International Conference on Information and Knowledge Management (CIKM), 2020.
- [7] Wang, Xuhong, Lyu, Dexing, Meng, Mengting, Yan, Xiaobing, and Ji, Yang. 'APAN: Asynchronous Propagating Attention Network for Real-time Temporal Graph Embedding.' In ACM SIGMOD International Conference on Management of Data, pp. 2628--2638, 2021.
- [8] Wang, Yanbang, Chang, Yen-Yu, Liu, Yunyu, Leskovec, Jure, and Shen, Pan. 'Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks.' In International Conference on Learning Representations (ICLR), 2021.
- [9] Wang, Lu, Chang, Xiaojun, Li, Shenghao, Chu, Yunfei, Li, Huan, and Wei, Zhewei. 'TCL: Temporal Contrastive Learning for Dynamic Graph Representation.' In Advances in Neural Information Processing Systems (NeurIPS), vol. 34, pp. 25010--25022, 2021.
- [10] Zhou, Fan and Cao, Chengtai. 'Overcoming Catastrophic Forgetting in Graph Neural Networks with Experience Replay.' In AAAI Conference on Artificial Intelligence (AAAI), vol. 35, pp. 4714--4722, 2021.
- [11] Xu, Yishi, Zhang, Yingxue, Guo, Wei, Guo, Huifeng, Tang, Ruiming, and Xiu, Mark. 'GraphSAIL: Graph Structure Aware Incremental Learning for Recommender Systems.' In ACM International Conference on Information and Knowledge Management (CIKM), pp. 1585--1594, 2020.
- [12] Liu, Junwei, Yang, Jialing, Song, Meng, Gao, Jing, and He, Xiangnan. 'Lifelong Graph Learning.' In IEEE International Conference on Data Mining (ICDM), pp. 380--389, 2021.
- [13] Zhu, Cunchao, Chen, Muhao, Fan, Changjun, Cheng, Qian, and Zhang, Yan. 'Learning from History: Modeling Temporal Knowledge Graphs with Copy-Generation Networks.' In AAAI Conference on Artificial Intelligence (AAAI), vol. 35, pp. 4741--4749, 2021.
- [14] Poursafaei, Farimah, Huang, Shenyang, Pelrine, Kellin, and Rabbany, Reihaneh. 'Towards Better Evaluation for Dynamic Link Prediction.' In Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 32928--32941, 2022.
- [15] Gao, Jianan, Zhao, Mengran, Song, Yang, and Zhang, Muhan. 'Handling Spatio-Temporal Distribution Shifts in Dynamic Graph Neural Networks.' In International Conference on Machine Learning (ICML), 2023.
- [16] Huang, Shenyang, Poursafaei, Farimah, Danovitch, Jacob, Fey, Matthias, Hu, Weihua, Rossi, Emanuele, Leskovec, Jure, Bronstein, Michael M., and Rabbany, Reihaneh. 'Temporal Graph Benchmark for Machine Learning on Dynamic Graphs.' In Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 14398--14421, 2023.

- [17] Cong, Weilin, Guo, Siheng, Kang, Jian, Chen, Boyu, and Zhou, Xiang. 'Do We Really Need Complicated Model Architectures for Temporal Networks?.' In International Conference on Learning Representations (ICLR), 2023.
- [18] Yu, Le, Sun, Leilei, Du, Bowen, and Lv, Weifeng. 'Towards Better Dynamic Graph Learning: New Architecture and Unified Library.' In Advances in Neural Information Processing Systems (NeurIPS), vol. 36, pp. 65888--65901, 2023.
- [19] Huang, Shenyang, Poursafaei, Farimah, Fey, Matthias, and Rabbany, Reihaneh. 'TGB 2.0: A Benchmark for Dynamic Node, Link, and Graph-Level Tasks on Large-Scale Temporal Graphs.' In Advances in Neural Information Processing Systems (NeurIPS), 2024.
- [20] Sankar, Aravind, Wu, Junshan, Yan, Xifeng, and Han, Jiawei. 'Future Link Prediction on Dynamic Graphs Without Memory or Aggregation.' In Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.
- [21] Zhang, Xikun, Song, Dongjin, and Tao, Dacheng. 'Continual Graph Learning: A Survey.' In IEEE Transactions on Knowledge and Data Engineering (TKDE), vol. 36, no. 8, pp. 3912--3931, 2024.
- [22] Li, Jiarui, Chen, Meng, Huang, Zhenke, and Zhao, Wayne Xin. 'TGFormer: Dynamic Graph Transformer with Long-Range Temporal Attention.' In ACM International Conference on Information and Knowledge Management (CIKM), 2023.
- [23] Zhang, Qiang, Liu, Jialing, Wu, Han, and Gao, Jing. 'NEU: Non-Volatile Memory-Enhanced Graph Representation Learning for Continuous-Time Networks.' In ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), pp. 2451--2460, 2022.
- [24] Gravina, Alessio, Bacciu, Davide, and Zambon, Daniele. 'Anti-Symmetric Dynamic Graph Neural Networks.' In IEEE Transactions on Neural Networks and Learning Systems, vol. 35, no. 4, pp. 4812--4825, 2024.
- [25] Panzarasa, Pietro, Opsahl, Tore, and Carley, Kathleen M. 'Patterns and Dynamics of Users' Behavior and Interaction: Network Analysis of an Online Community.' In Journal of the American Society for Information Science and Technology, vol. 60, no. 5, pp. 911--932, 2009. [Online: <https://snap.stanford.edu/data/CollegeMsg.html>]
- [26] Kumar, Srijan, Spezzano, Francesca, Subrahmanian, V. S., and Faloutsos, Christos. 'Edge Weight Prediction in Weighted Signed Networks.' In IEEE International Conference on Data Mining (ICDM), pp. 221--230, 2016. [Online: <https://snap.stanford.edu/data/soc-sign-bitcoin-otc.html>]
- [27] Holland, Paul W., Laskey, Kathryn Blackmond, and Leinhardt, Samuel. 'Stochastic blockmodels: First steps.' In Social Networks, vol. 5, no. 2, pp. 109--137, 1983.
- [28] Kazemi, Seyed Mehran, Goel, Rishab, Jain, Kshitij, Karki, Shirish, Gabaldon, Noah, Amer, Reihaneh, Rabbany, Reihaneh, and Perkins, Colin. 'Representation Learning for Dynamic Graphs: A Survey.' In Journal of Machine Learning Research (JMLR), vol. 21, no. 70, pp. 1--73, 2020.
- [29] Kou, Chao, Hou, Tingting, Wang, Xiang, and He, Xiangnan. 'Continual Graph Learning with Experience Replay.' In Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [30] Kirkpatrick, James, Pascanu, Razvan, Rabinowitz, Neil, Veness, Joel, Desjardins, Guillaume, Rusu, Andrei A., Milan, Kieran, Quan, John, Ramalho, Tiago, Grabska-Barwinska, Agnieszka, and others. 'Overcoming Catastrophic Forgetting in Neural Networks.' In Proceedings of the National Academy of Sciences (PNAS), vol. 114, no. 13, pp. 3521--3526, 2017.
- [31] Chen, X., Zhao, L., Sun, Y., and Wang, H. 'Towards Robust Temporal Graph Neural Networks: Mitigating Memory Interference and Representation Drift.' In IEEE Transactions on Neural Networks and Learning Systems (TNNLS), vol. 37, no. 2, pp. 1120--1134, 2025.
- [32] Huang, Shenyang, Poursafaei, Farimah, Fey, Matthias, and Rabbany, Reihaneh. 'Benchmarking Dynamic Link Prediction under Regime Recurrence and Structural Shifts.' In Advances in Neural Information Processing Systems (NeurIPS), vol. 38, pp. 21050--21068, 2025.
- [33] Zhao, Mengran, Gao, Jianan, Song, Yang, and Zhang, Muhan. 'Lifelong Dynamic Link Prediction under Multi-Phase Interaction Regimes.' In Proceedings of the AAAI Conference on Artificial Intelligence (AAAI), vol. 39, no. 14, pp. 15420--15428, 2025.
- [34] Liu, Xiang, Zhang, Wei, and He, Lifang. 'Continual Learning on Dynamic Graphs via Structural Memory Consolidation and Episodic Replay.' In ACM Transactions on Knowledge Discovery from Data (TKDD), vol. 19, no. 3, pp. 1--24, 2025.
- [35] Wang, Haohui, Chen, Yue, and Leskovec, Jure. 'Recurrent and Memory-Augmented Graph Representation Learning on Non-Stationary Temporal Streams.' In International Conference on Learning Representations (ICLR), 2025.
- [36] Zhang, Yulong, Tang, Jiliang, and Bronstein, Michael M. 'A Survey on Temporal Graph Neural Networks: Foundations, Dynamics, and Open Frontiers.' In ACM Computing Surveys (CSUR), vol. 58, no. 1, pp. 1--38, 2026.

Supplementary Material & Appendices

Appendix A: Extended Hyperparameter Protocol

The reported models use Adam with $\beta_1=0.9$, $\beta_2=0.999$, learning rate 0.005, and weight decay $1e-4$ for 10 epochs. The canonical checkpoint interval is 10 snapshots, $K=10$, and $d_m=64$. The reproducibility record should also include the validation split, any early-stopping rule, tuned hyperparameters, negative-sampling ratio, and search budget for each baseline.

Appendix B: Eight-Point Non-Anticipation Audit Protocol

The eight-point audit was applied to every reported run: candidate-edge parity, label parity, strict temporal masking, post-evaluation memory updates, feature and node-identity checks, checkpoint masking, regime-boundary blindness, and deterministic shared negative generation. These conditions are suitable for automated assertions in the released implementation.

Appendix C: Analytical Memory Derivations

Analytical RAM formula for episodic memory bank storage:

$$M_{RAM} = 4 * (N * d_m + K * d_k + K * N * d_m) / 1024 \text{ (in KB)}$$

For $N=300$, $d_m=64$, $d_k=64$, $K=10$, this yields $4 * (19,200 + 640 + 192,000) / 1024 = 827.5$ KB. Using the same calculation with $N=10,000$ and $K=20$ gives an estimated bank size of 52.5 MB.

Appendix D: Real-World Episode Selection Protocol

The real-world analysis uses exploratory multi-week episodes from SNAP CollegeMsg (<https://snap.stanford.edu/data/CollegeMsg.html>) and Bitcoin-OTC (<https://snap.stanford.edu/data/soc-sign-bitcoin-otc.html>) [25, 26]. The reproducibility record should state the community-detection method, label-alignment procedure, overlap threshold, and treatment of overlapping CollegeMsg windows. Bitcoin-OTC is a signed trust network. We therefore do not assign market-cycle labels unless an external price series and a clearly defined segmentation rule are available.

Appendix E: Early Benchmark Prototypes and Failure Modes

Early Erdős-Rényi prototypes allowed density to change from $\rho_A=0.20$ to $\rho_B=0.05$. That made the regime transition easy to detect from density alone. The canonical benchmark removes that shortcut by keeping $\rho=0.10$ across regimes.